

Evolutionary Computing

Grundlagen Optimierung

Prof. Dr. Veronika Lesch

INF23

Copyright

Alle Materialien in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Vorlesungs- und Übungsmaterialien an Dritte ist generell untersagt!

- 1 Einführung Optimierung
- 2 Formale Grundlagen der Optimierung
- 3 Eigenschaften von Problemen
- 4 Naive Suchverfahren
- 5 Quellen

Einführung Optimierung

Sie haben 15min Zeit für Ihr Frühstück und folgende Optionen:

- Toast: 3min (Sättigung 2)
- Müsli: 5min (Sättigung 4)
- Smoothie: 10min (Sättigung 6)

Was sollten Sie frühstücken um Ihre Sättigung zu maximieren?

Die Supermarkt Warteschlange

Sie stehen im Supermarkt und gehen zur Kasse. Es sind 2 Kassen geöffnet:

- Kasse A: 2 Personen mit vollem Einkaufswagen
- Kasse B: 6 Personen mit wenigen Artikeln

Ein Bezahlvorgang für einen vollen Wagen dauert im Schnitt 2 Minuten. Für wenige Artikel benötigt der Bezahlvorgang im Schnitt 30 Sekunden.

Welche Kasse sollten Sie wählen?

Der Bauer und sein Zaun

Ein Bauer hat 100m Zaun zur Verfügung und möchte damit ein Rechteck einzäunen.

Welche Seitenlängen sollte das Rechteck haben, um die eingeschlossene Fläche zu maximieren?

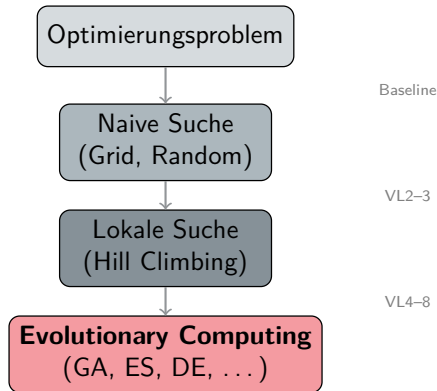
Warum beschäftigen wir uns mit Optimierung?

Optimierung findet die beste Entscheidung unter gegebenen Bedingungen.

Auswahl realer Anwendungen:

- Routenplanung (UPS, DHL, Amazon)
- Produktions- & Kapazitätsplanung
- Personalplanung / Schichtplanung
- Portfolio-Optimierung (Finance)
- Hyperparameter-Tuning in ML
- Chip-Design & VLSI-Layout
- Proteinstrukturvorhersage
- Energienetzsteuerung (Smart Grid)

Wo steht Optimierung im Kontext von EC?



Formale Grundlagen der Optimierung

Besprechen Sie in 10 Minuten in Partnerarbeit

- Welche **3 Situationen** haben Sie zuletzt “optimiert”?
- Bereiche: privat · Studium · Job/Nebenjob

Impulse: günstigste Bahnverbindung · Wohnungssuche · Schichtplan · Klausurvorbereitung

Gemeinsamkeiten aller Beispiele

Gemeinsame Eigenschaften:

- 1 Etwas, das entschieden werden muss
- 2 Ein Ziel, das es zu erreichen gilt
- 3 Einschränkungen, die gelten müssen

Formalisierung:

- “Was entscheide ich?” → Variablen
- “Wie gut ist die Entscheidung?” → Zielfunktion
- “Was ist erlaubt?” → Nebenbedingungen

Jedes reale Optimierungsproblem lässt sich in dieses dreiteilige Schema übersetzen.

Formale Definition Optimierungsproblem [4]

$$P : \quad \min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{x} \in \mathcal{M}$$

$$\mathcal{M} = \{\mathbf{x} \in \mathcal{X} \mid g_i(\mathbf{x}) \leq 0, \quad h_j(\mathbf{x}) = 0, \quad i \in \mathcal{I}, j \in \mathcal{J}\}$$

- $f : \mathcal{X} \rightarrow \mathbb{R}$: **Zielfunktion** (Fitness, Objective)
- **Zulässige Menge** $\mathcal{M}$: Menge aller zulässigen Lösungen
- $\mathbf{x} \in \mathcal{X}$: **Entscheidungsvektor** im Suchraum $\mathcal{X}$
- g_i, h_j : **Nebenbedingungen** (Ungleichung / Gleichung)
- **Globales Optimum**:
 $\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathcal{M}} f(\mathbf{x})$
- **Lokales Optimum**: besser als alle direkten Nachbarn

Beispiel - Das Rucksack Problem (Knapsack)

Problemstellung

Ein Dieb steht vor einem Tresor mit 4 Gegenständen (siehe Tabelle unten). Sein Rucksack trägt maximal das Gewicht 8. Welche Gegenstände soll er einpacken, um den **größtmöglichen Gesamtwert** mitzunehmen?

Gegenstand	Gewicht w_i	Wert v_i
1	2	3
2	3	4
3	4	5
4	5	6

Rucksack-Problem: Formalisierung

Suchraum	$\mathcal{X} = \{0, 1\}^n, \quad x_i \in \{0, 1\}$	Binärer Vektor: Gegenstand i eingepackt (1) oder nicht (0)
Zielfunktion	$f(x) = - \sum_{i=1}^n v_i x_i$	Negativer Gesamtwert der gewählten Gegenstände (Minimierung)
Ungleichungs- nebenbedingung	$g_1(x) = \sum_{i=1}^n w_i x_i - 8 \leq 0$ $(\mathcal{I} = 1)$	Gesamtgewicht darf die Kapazität 8 nicht überschreiten
Gleichungs- nebenbedingungen	$\mathcal{J} = \emptyset$ $(\mathcal{J} = 0)$	Keine Gleichungsnebenbedingungen vorhanden

Optimierung vs. Constraint Satisfaction [3]

	Zielfunktion	
Nebenbedingungen	Ja	Nein
Ja	Constrained optimisation problem (COP)	Constraint satisfaction problem (CSP)
Nein	Free optimisation problem (FOP)	Kein Problem

Besprechen Sie in 10 Minuten in Partnerarbeit

- Welche Problemklasse hat das eben besprochene Rucksack Problem?
- Wie müsste die Problemstellung für die anderen 3 Problemklassen lauten?

Eigenschaften von Problemen

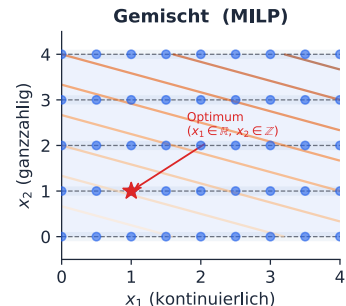
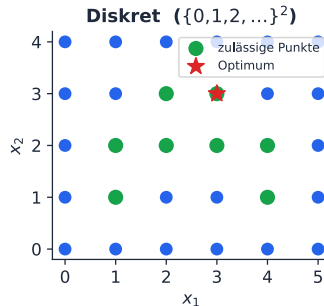
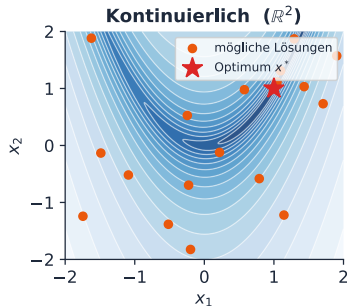
Showcase - Manuelle Optimierung



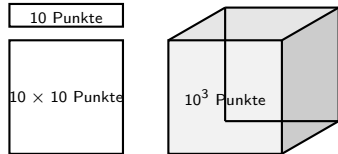
[1]

1. Suchraum

Der Suchraum $\mathcal{X}$ ist die Menge *aller potenziellen* Lösungen (feasible und infeasible).



Beobachtung [2]: Das Volumen eines hochdimensionalen Raumes wächst so schnell, dass jede endliche Stichprobe darin äußerst dünn wird.

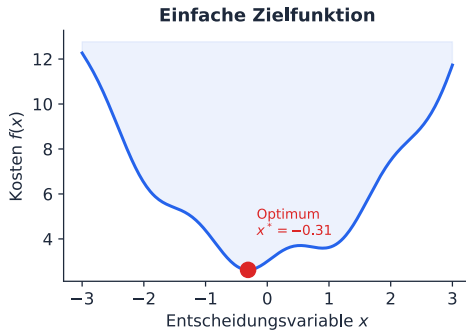


Suchraum	Formel	Beispiel	Größenordnung
Kontinuierlich	$\mathbf{x} \in \mathbb{R}^n$	$n = 10$ Variablen	∞
Binärraum	$ \mathcal{X} = 2^n$	$n = 50$ Bits	$\approx 10^{15}$
Binärraum	$ \mathcal{X} = 2^n$	$n = 100$ Bits	$\approx 10^{30}$
Permutationen (TSP)	$ \mathcal{X} = n!$	$n = 20$ Städte	$\approx 10^{18}$
Permutationen (TSP)	$ \mathcal{X} = n!$	$n = 50$ Städte	$\approx 10^{64}$

2. Zielfunktion

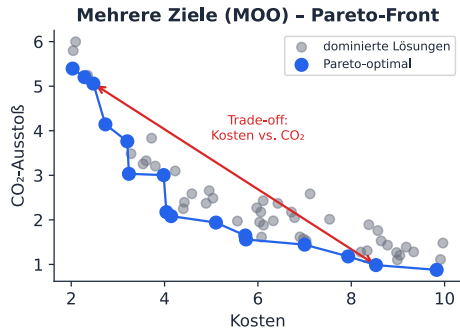
Einfach (Single-Objective)

Genau eine Kennzahl wird minimiert (z.B. Kosten). Eindeutiges Optimum.



Mehrfach (MOO)

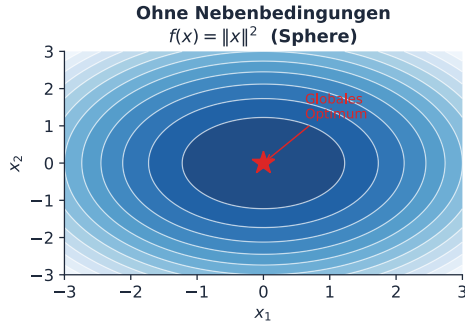
Mehrere Ziele *gleichzeitig* – z.B. Kosten und CO₂. Ziele widersprechen sich oft.



3. Nebenbedingungen

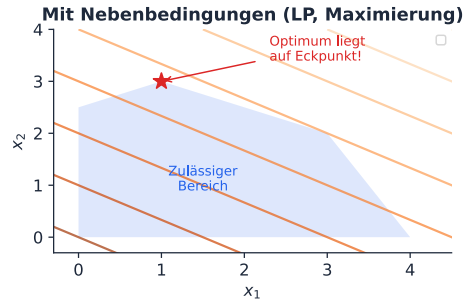
Unconstrained

Jeder Punkt im Suchraum ist gültig. Viele Benchmark-Funktionen (z.B. Sphere) sind so definiert. Gradient zeigt direkt ins Optimum.



Constrained (mit NB)

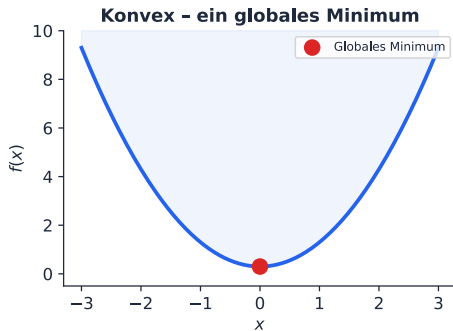
Nur Punkte im zulässigen Bereich sind erlaubt. Das Optimum liegt oft auf dem Rand des Bereichs.



4. Struktur

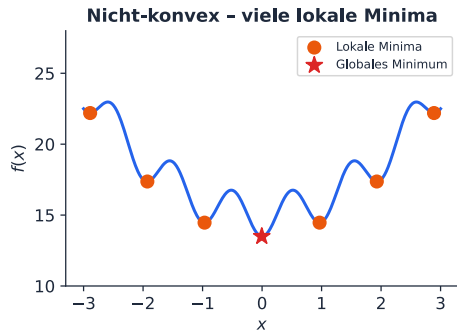
Konvex

Jeder lokale Abstieg führt garantiert zum globalen Minimum.



Nicht-konvex

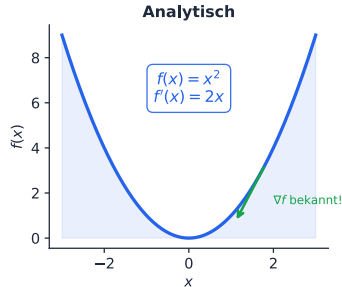
Viele lokale Minima – lokale Suche bleibt stecken.



5. Auswertung

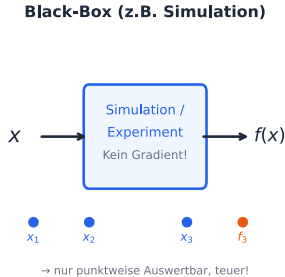
Analytisch

Geschlossene Formel,
 Gradient direkt berechenbar.



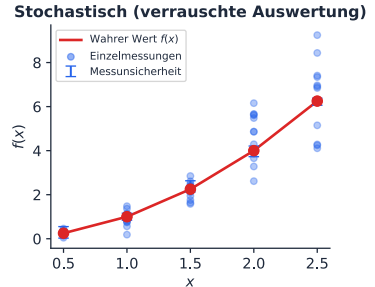
Black-Box

Kein Einblick in f .
 Nur Input $\rightarrow$ Output.



Stochastisch

Gleicher Input x liefert
 verschiedene Outputs.



Zusammenfassung: Eigenschaften im Überblick

Eigenschaft	Einfacher Fall	Schwieriger Fall	Lösungsansatz
Suchraum	kontinuierlich, diskret	MILP (gemischt)	Branch & Bound, GA
Zielfunktion	Einfach	MOO	NSGA-II, Pareto
Nebenbedingungen	Keine	Viele NB	Strafterme, LP
Struktur	Konvex	Nicht-konvex	Restarts, SA, GA
Auswertung	Analytisch	Black-Box/Stoch.	Bayes. Opt., CMA-ES

**Je mehr schwierige Eigenschaften ein Problem hat, desto eher sind
Metaheuristiken eine vielversprechende Wahl.**

Naive Suchverfahren

Showcase - Manuelle Optimierung



[1]

Naive Suchverfahren als Baseline

Ohne Baseline lässt sich kein Verfahren sinnvoll bewerten. Naive Suche ist die klassische Baseline: keinerlei Problem-Wissen, maximale Allgemeinheit.

Eigenschaften naiver Suchverfahren:

- **Einfach:** wenige Parameter, leicht implementierbar
- **Robust:** keinerlei Annahmen über f oder $\mathcal{X}$
- **Parallelisierbar:** Samples sind unabhängig
- **Naiv:** kein Lernen, keine Anpassung
- **Baseline:** Jeder spätere Algorithmus muss besser sein – sonst lohnt er sich nicht

Exhaustive Search (Brute Force)

Für alle $\mathbf{x} \in \mathcal{X}$: bewerte $f(\mathbf{x})$, merke $\mathbf{x}^* = \arg \min f$

Eigenschaften:

- Garantiert globales Optimum
- Korrektheit durch vollständige Enumeration
- Zeitkomplexität: $\mathcal{O}(|\mathcal{X}|)$

Grenzen:

- Suchraum muss endlich und aufzählbar sein
- Für große $\mathcal{X}$ praktisch unmöglich
- Für kontinuierliche Problemen nicht anwendbar

Exhaustive Search ist nur für sehr kleine Probleme praktikabel und dient als theoretische Referenz.



[1]

- Problemklassen identifizieren
- Formalisierung von Problemen
- Brute Force Rucksackproblem
- Das 8-Damen Problem

Quellen

- [1] Generiert von Microsoft Copilot.
[Online; accessed 2026-03-12].

- [2] Richard Bellman.
Dynamic Programming.
Princeton University Press, Princeton, NJ, 1957.

- [3] A. E. Eiben and J. E. Smith.
Introduction to Evolutionary Computing.
Natural Computing Series. Springer, Berlin, Heidelberg, 2 edition, 2015.

- [4] Nathan Sudermann-Merx.
Einführung in Optimierungsmodelle.
Springer Spektrum Berlin, Heidelberg, 2023.